

Shake & Snack: A Multi-Screen Food Picker App Built With MIT App Inventor

Sujay Tailor

Durham College

INFT3101: MIT App Inventor – App Development Lab 2

Mohammad Shamas

September 23, 2026

Table of Contents

Contents

Introduction.....	2
App Design.....	2
Screens.....	2
Input Types and Sensors.....	2
Personalization.....	3
Blocks Code.....	3
Screen1 Blocks.....	3
PickerScreen Blocks.....	4
App Running on Device.....	7
Screen 1: Welcome.....	8
Screen 2: Pick My Meal.....	9
Testing.....	10
Bugs Found and Fixed.....	11
Reflection.....	11
Experience.....	11
Challenges.....	12
What I Learned.....	12
Conclusion.....	12

References.....	14
-----------------	----

Shake & Snack: A Multi-Screen Food Picker App Built With MIT App Inventor

Introduction

Shake & Snack is an Android app built with MIT App Inventor (Massachusetts Institute of Technology [MIT], n.d.-a). It is made for people who cannot decide what to eat. The user types their name on a welcome screen and then moves to a second screen, where they choose a cuisine, set a budget with a slider and tick diet options. When they tap a button or shake the phone, the app picks a random meal that fits the cuisine and diet choice, shows a photo of that cuisine and reads the result out loud with text-to-speech.

App Design

Screens

The app has two screens. Screen1 is the welcome screen. It shows the logo, the app name and a tagline, and it asks for the user's name.

PickerScreen is the feature screen, where all the meal picking happens.

Input Types and Sensors

Table 1 lists every input type, sensor and output used in the app. The app uses six of the input types from the lab list, which is more than the five that were required.

Table 1

Input Types, Sensors and Components Used in Shake & Snack

Requirement	Component(s)	Use in the app
Text input	TextBox (txtName)	User's name
Buttons	btnStart, btnPick, btnSpeak, btnBack	Navigation and actions
Check boxes	chkVeg, chkSpicy	Vegetarian and spicy options
Dropdown menu	ListPicker (lpCuisine)	Choose Italian, Indian, Mexican, Chinese or Japanese
Slider	sldBudget	Budget from \$5 to \$40
Image display	imgLogo, imgFood	Logo and a photo of the chosen cuisine
Sensor	AccelerometerSensor1	Shake the phone to pick a meal
External interaction	TextToSpeech1	Reads the result out loud
Supporting	Notifier1	Alert when the name box is empty

Personalization

The app uses a warm, food-themed colour palette: orange (#FF7043), dark orange (#E64A19), cream (#FFF3E0) and brown (#4E342E). It has a custom logo that is also used as the app icon, and a photo for each cuisine. The greeting and the spoken result both use the user's name. Every colour

was set by hand so the app looks the same whether the phone is in light mode or dark mode. The checkboxes are centred and the two bottom buttons are spaced apart so the screen is easy to use.

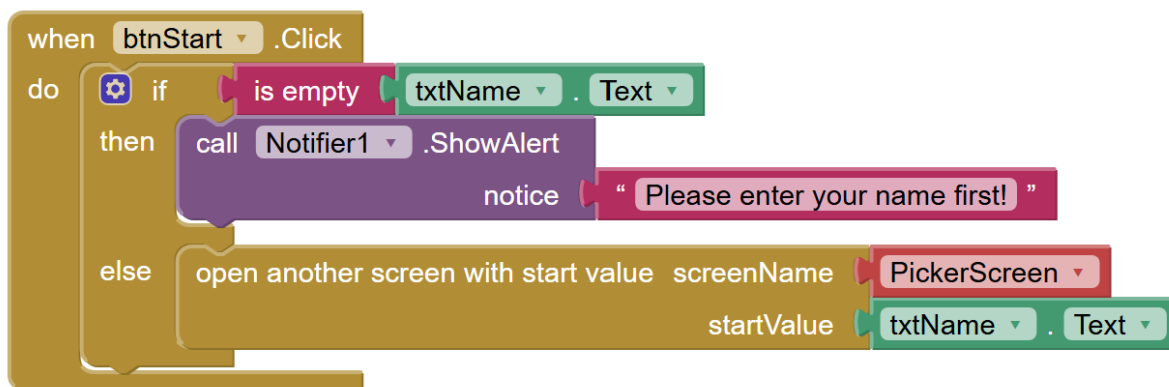
Blocks Code

Screen1 Blocks

Figure 1 shows the only event on Screen1. When the Let's Eat! button is tapped, the app checks if the name box is empty. If it is, a notifier asks the user to enter their name. If not, the app opens PickerScreen and sends the name along as the start value.

Figure 1

Screen1 Blocks for the Let's Eat! Button

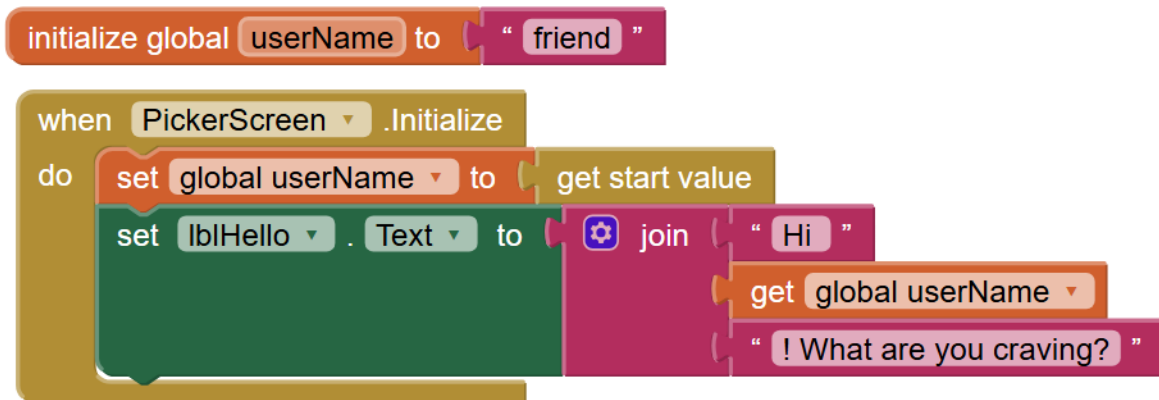


PickerScreen Blocks

Figure 2 shows the global variable and the start-up event. When PickerScreen opens, it saves the name from Screen1 in the userName variable and shows a greeting such as "Hi Sujay! What are you craving?"

Figure 2

Global Variable and PickerScreen Start-Up Event



Figures 3 and 4 show the pickMeal procedure, which is the main logic of the app. It checks which cuisine was chosen, then uses the Vegetarian checkbox to decide between a vegetarian list and a non-vegetarian list, and picks a random meal from that list. After that it shows the cuisine photo and the budget, adds “extra spicy!” when Spicy is ticked, and speaks the result.

Figure 3

The pickMeal Procedure, Part 1 (Italian, Indian and Mexican)

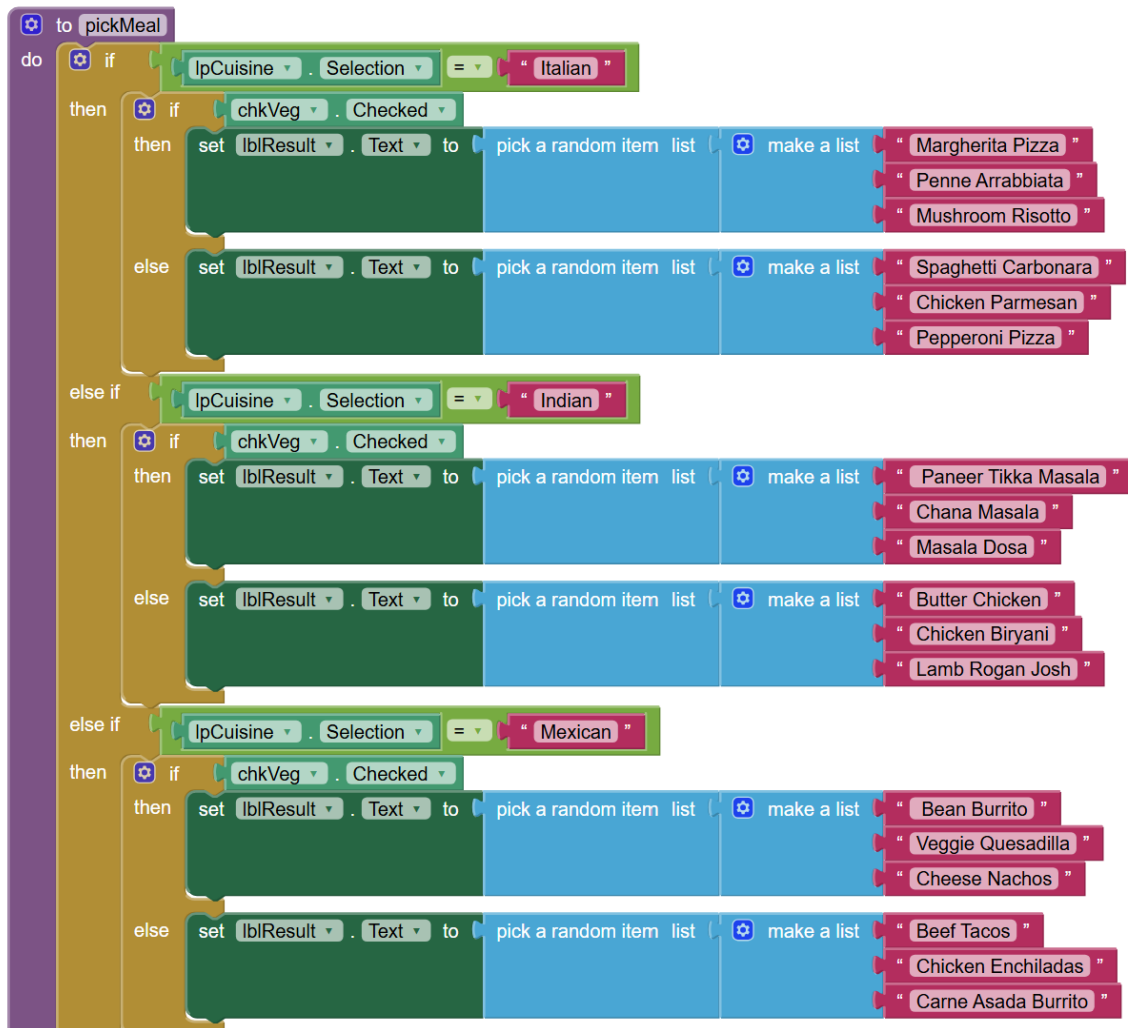


Figure 4

The pickMeal Procedure, Part 2 (Chinese, Japanese, Photo, Details and Speech)

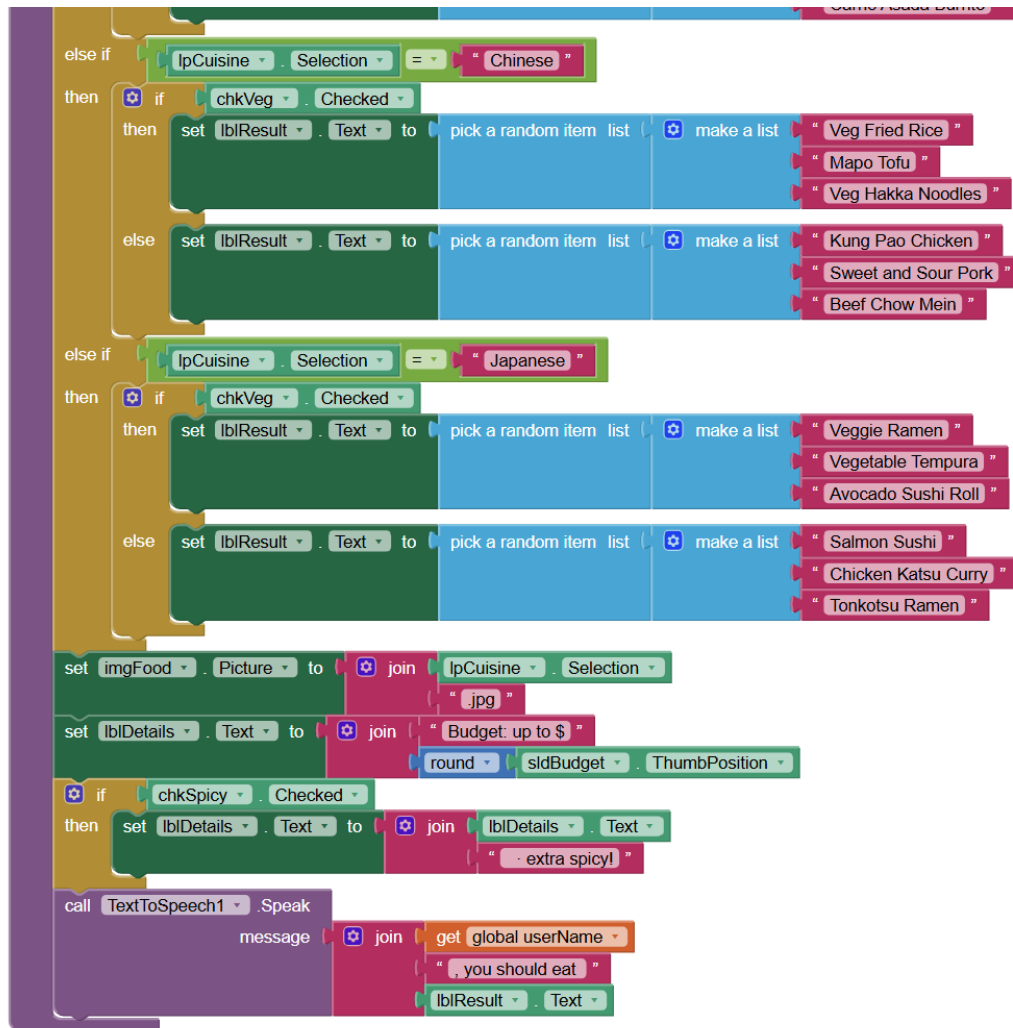


Figure 5 shows the slider and cuisine picker events. Moving the slider updates the budget label with a rounded number, and choosing a cuisine updates the text on the picker button.

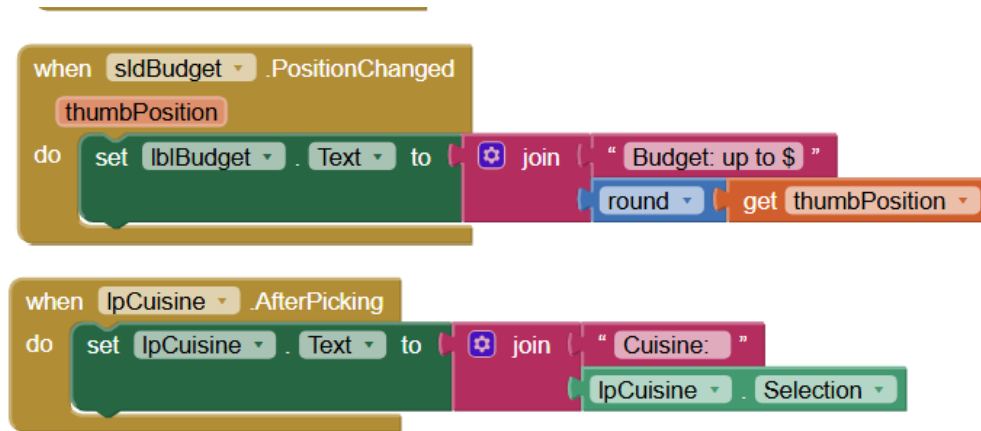
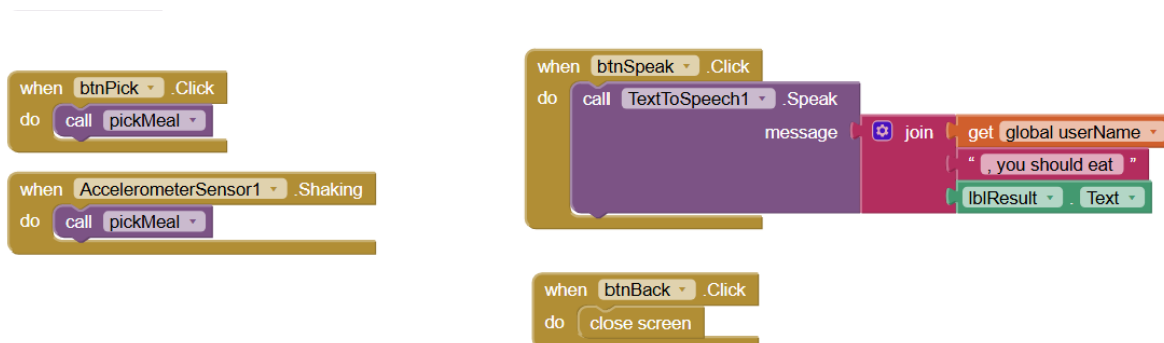
Figure 5*Slider and Cuisine Picker Events*

Figure 6 shows the button and sensor events. Tapping Pick for me! or shaking the phone both call pickMeal, so the same code is reused. Say it again repeats the spoken result, and Back closes PickerScreen and returns to Screen1.

Figure 6*Button and Accelerometer Events*

App Running on Device

The final version of the app was installed as an .apk file and tested on a Samsung Galaxy S25 Ultra running Android 16. During development it was also tested live on an iPhone with the MIT AI2 Companion app (MIT, n.d.-b). Figures 7 to 11 were taken on the Samsung phone.

Screen 1: Welcome

Figure 7

Welcome Screen

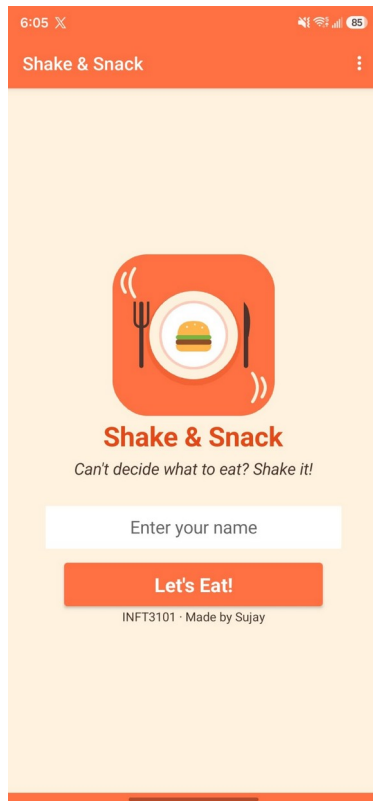
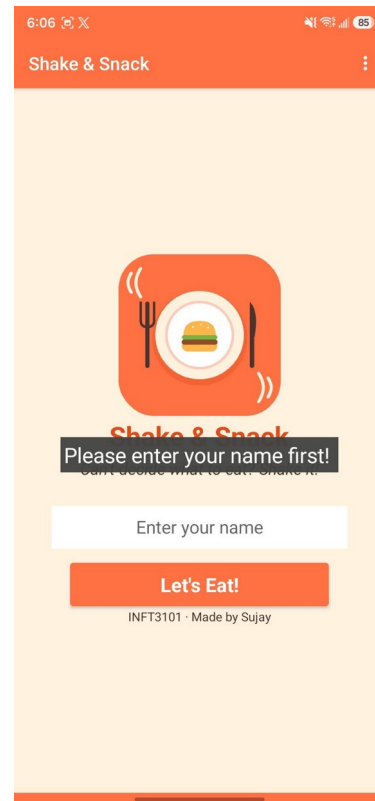


Figure 8

Alert When No Name Is Entered



Screen 2: Pick My Meal

Figure 9

PickerScreen Greeting the User by Name

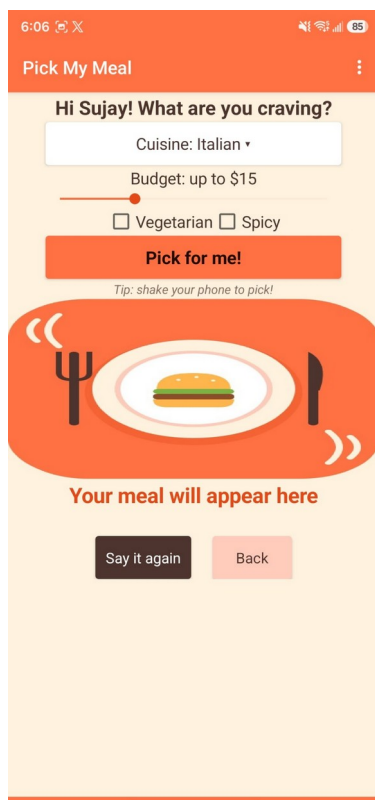


Figure 10

Vegetarian Italian Result

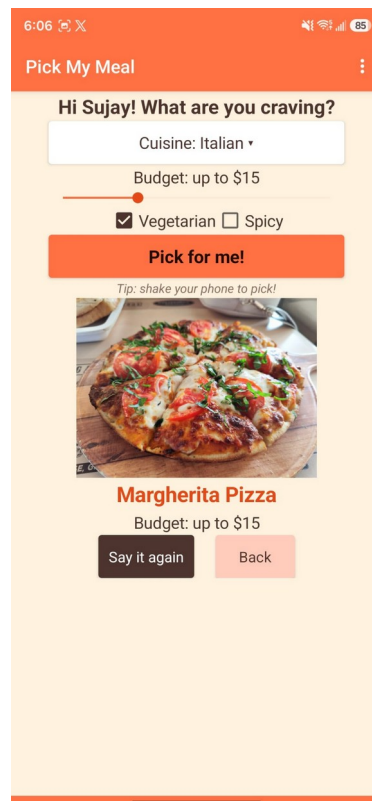
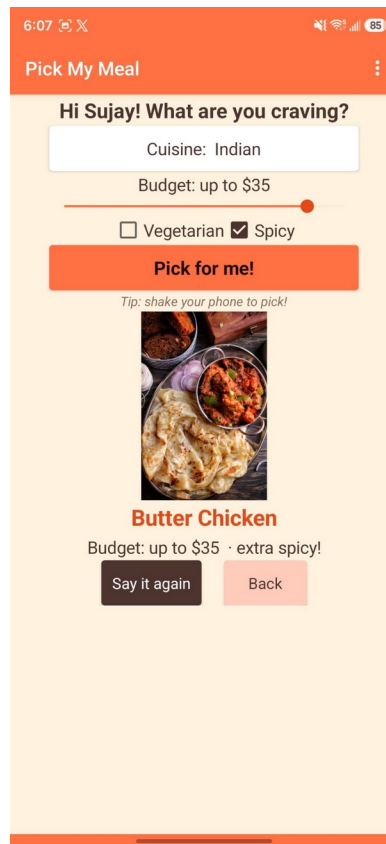


Figure 11

Non-Vegetarian Indian Result With Spicy Option and Higher Budget



Testing

Each feature was tested on the Samsung phone. Table 2 shows the test cases and results.

Table 2

Test Cases and Results

#	Test	Expected result	Result
1	Tap Let's Eat! with an empty name	Alert is shown	Pass
2	Enter name and tap Let's Eat!	PickerScreen opens with greeting	Pass

#	Test	Expected result	Result
3	Move the slider	Budget label updates	Pass
4	Choose a cuisine in the picker	Button shows “Cuisine: ...”	Pass
5	Italian + Vegetarian, tap Pick for me!	Vegetarian Italian meal, photo and speech	Pass
6	Indian + Spicy, tap Pick for me!	Non-vegetarian Indian meal with “extra spicy!”	Pass
7	Shake the phone	A new meal is picked	Pass
8	Tap Say it again	Last result is spoken again	Pass
9	Tap Back	Returns to Screen1	Pass

Bugs Found and Fixed

- The Let’s Eat! button did nothing on the Android phone because Screen1 had no blocks yet. Adding the btnStart.Click event fixed it.
- The photo, budget and speech blocks first snapped inside the Japanese section, so they only ran for Japanese food. Moving them below the big if block fixed it.
- On a phone in dark mode the background turned dark and some text was hard to read. Setting every colour by hand fixed it.
- In the iPhone Companion, the greeting showed “Hi !” because the name was not passed between screens. The installed Android app shows the name correctly.

Reflection

Experience

This was my first time building an app with block-based programming, and I liked how fast I could see results. Dragging a block onto the screen and

seeing the change on my phone a second later through the Companion app made it feel less like coding and more like building with pieces. My favourite moment was the first time the app said “Sujay, you should eat Butter Chicken” out loud after I shook the phone. The Designer was also easy to use once I understood that each component has its own properties panel.

Challenges

The hardest part was getting blocks to snap into the right place. At one point the photo and speech blocks ended up inside the Japanese section of my if block, so they only worked for one cuisine, and it took me a while to notice because the blocks looked fine. Dark mode was another surprise. My phone uses a dark theme, so the app looked brown instead of cream, and I learned that I had to set every colour myself so it would look the same on any phone. Image names also caused problems, because the photo only showed up when the file name matched the cuisine name exactly, including the capital letter and the .jpg ending. Finally, when I installed the app on an Android phone, the Let’s Eat! button did nothing, and I realised I had built all the blocks for the second screen but forgot the block for the first one.

What I Learned

I learned how event-driven programming works, where the app waits for something to happen, like a tap, a slider move or a shake, and then runs the blocks for that event. I also learned why procedures are useful. My pickMeal procedure is used by both the button and the accelerometer, so I only had to write the logic once. I got practice with if and else if blocks, lists,

random picks, passing data between screens and using a phone sensor. Testing on real devices taught me that an app can behave differently on an iPhone Companion, an Android phone, and light or dark mode, so it has to be tested on more than one setup. If I kept working on this app, I would give each meal its own photo, save favourite meals with TinyDB, and use the location sensor to suggest nearby restaurants.

Conclusion

Shake & Snack meets all of the lab requirements. It has two screens, six input types (text box, buttons, check boxes, a dropdown list picker, a slider and image display), an accelerometer sensor and text-to-speech output, all in a personalised food theme. Building it gave me hands-on practice with multiple screens, event-driven blocks, procedures and phone sensors in MIT App Inventor.

References

Massachusetts Institute of Technology. (n.d.-a). *MIT App Inventor* [Web application]. <https://appinventor.mit.edu>

Massachusetts Institute of Technology. (n.d.-b). *MIT AI2 Companion* [Mobile app]. <https://appinventor.mit.edu/explore/ai2/setup>

Pexels. (n.d.). *Free stock photos* [Photographs]. <https://www.pexels.com>